CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 5: Doubly Linked List and Circular Linked List Operations

Practical Aim: To implement core doubly linked list and circular linked list operations and understand how bidirectional and circular linking change the way nodes are inserted, deleted, and traversed.

Problem 1:

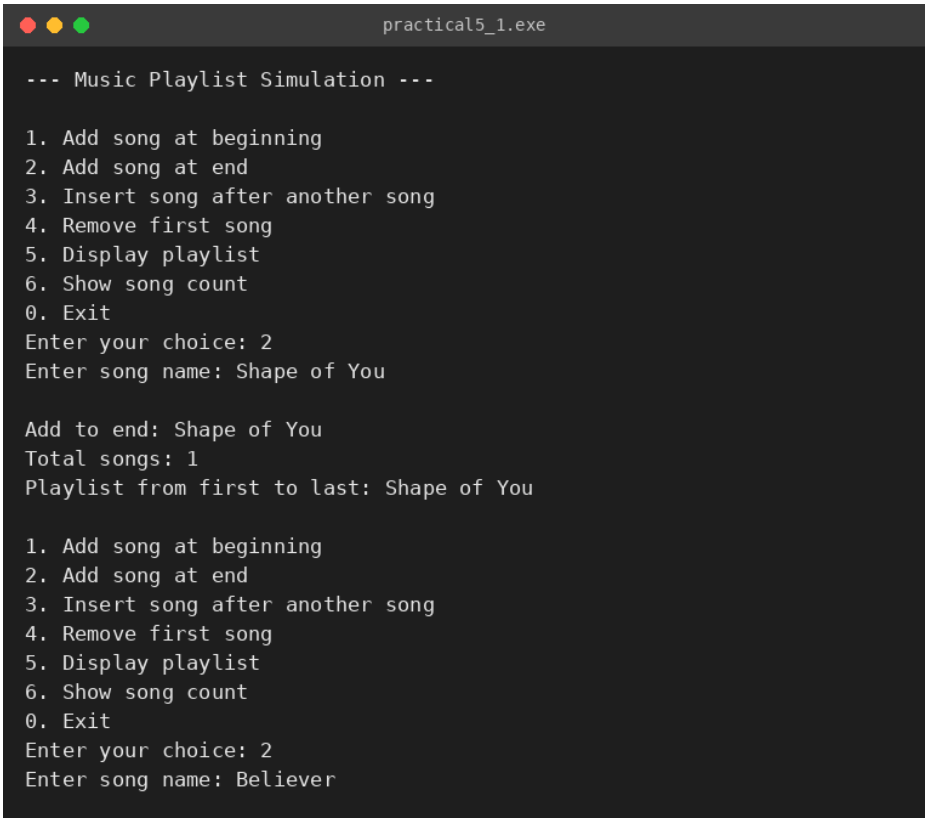
A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation.

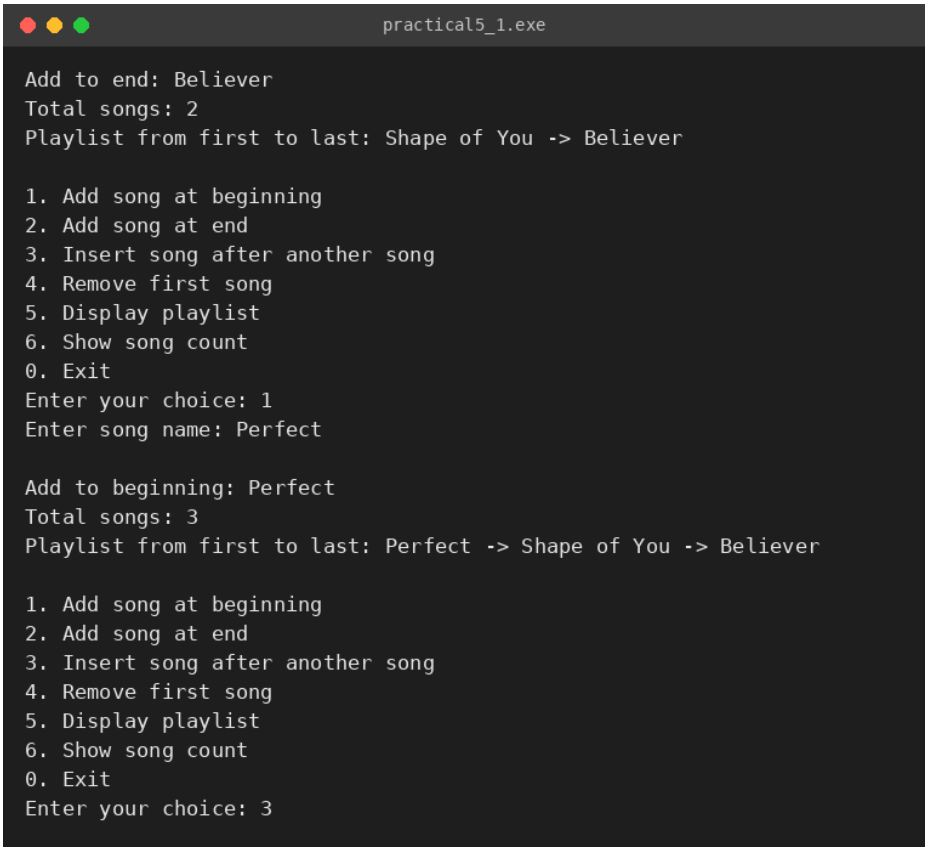
Approach: Each node stores the song name, a prev pointer and a next pointer; the list keeps head, tail and a song count. To insert after a given song, the list is searched from the head until the song is found. The new node is linked to its neighbours (newNode->prev = current, newNode->next = current->next), then the old next node's prev is updated to point back to the new node, and finally current->next is set to the new node. A singly linked list never has to repair the backward link of the following node, and the tail pointer must also be updated when the new song becomes the last one. If the given song is not found, the program prints "Song '<name>' not found. Insertion cancelled." and nothing is changed.

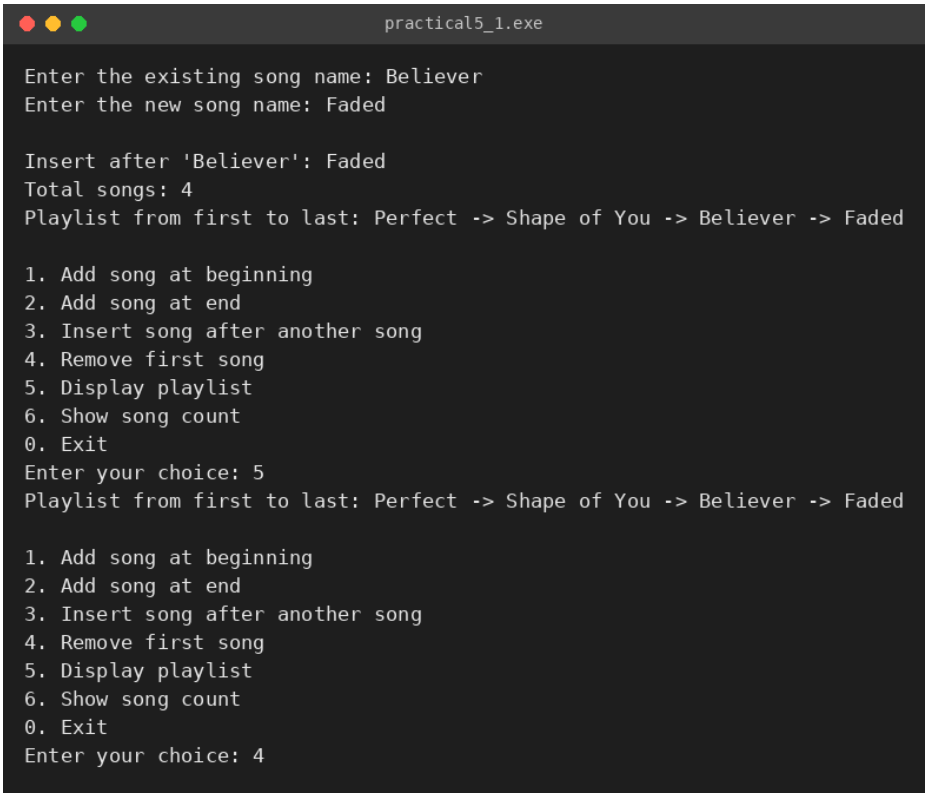
For Code:

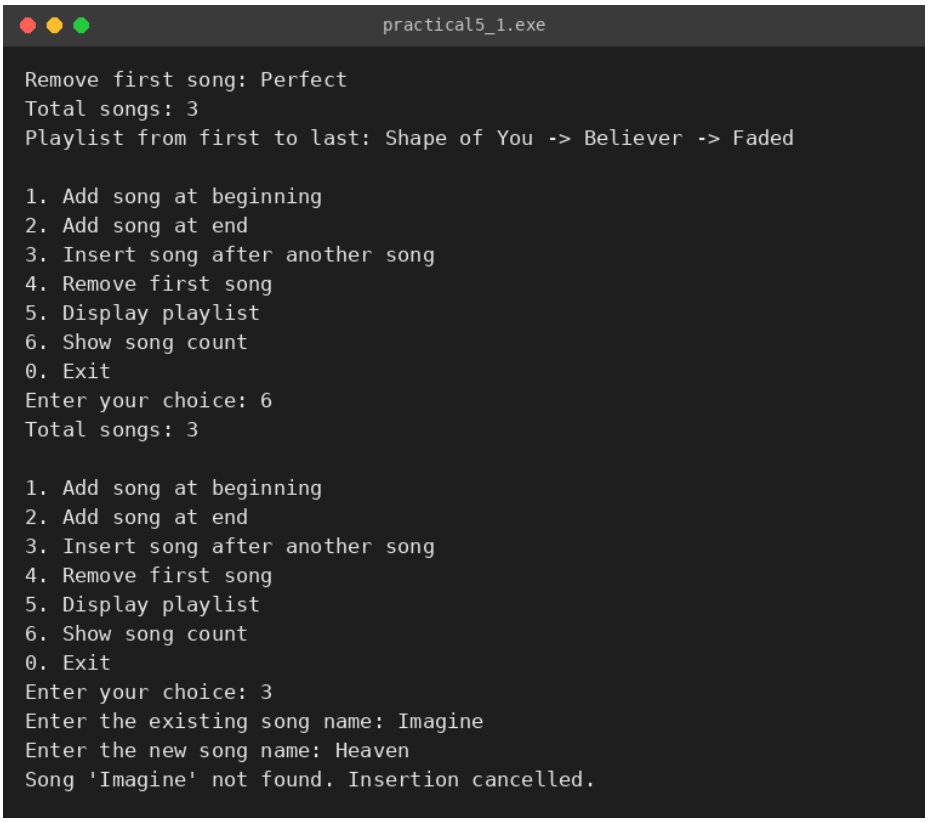
https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-5/practical5_1.cpp

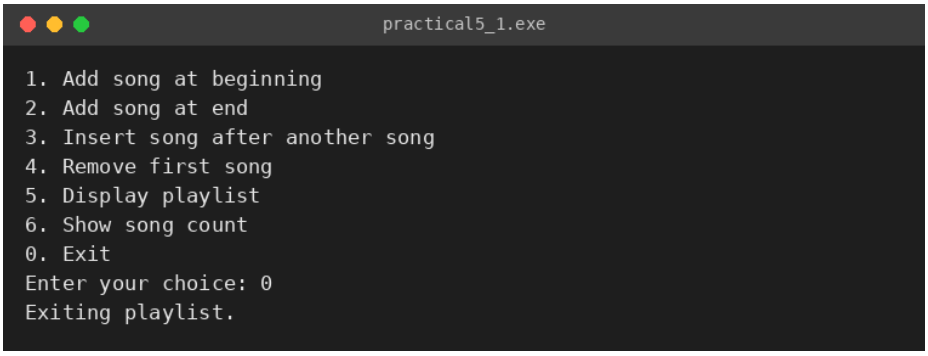
Output :


practical5_1.exe
--- Music Playlist Simulation ---
1. Add song at beginning
2. Add song at end
3. Insert song after another song
4. Remove first song
5. Display playlist
6. Show song count
0. Exit
Enter your choice: 2
Enter song name: Shape of You
Add to end: Shape of You
Total songs: 1
Playlist from first to last: Shape of You
1. Add song at beginning
2. Add song at end
3. Insert song after another song
4. Remove first song
5. Display playlist
6. Show song count
0. Exit
Enter your choice: 2
Enter song name: Believer


practical5_1.exe
Add to end: Believer
Total songs: 2
Playlist from first to last: Shape of You -> Believer
1. Add song at beginning
2. Add song at end
3. Insert song after another song
4. Remove first song
5. Display playlist
6. Show song count
0. Exit
Enter your choice: 1
Enter song name: Perfect
Add to beginning: Perfect
Total songs: 3
Playlist from first to last: Perfect -> Shape of You -> Believer
1. Add song at beginning
2. Add song at end
3. Insert song after another song
4. Remove first song
5. Display playlist
6. Show song count
0. Exit
Enter your choice: 3


practical5_1.exe
Enter the existing song name: Believer
Enter the new song name: Faded
Insert after 'Believer': Faded
Total songs: 4
Playlist from first to last: Perfect -> Shape of You -> Believer -> Faded
1. Add song at beginning
2. Add song at end
3. Insert song after another song
4. Remove first song
5. Display playlist
6. Show song count
0. Exit
Enter your choice: 5
Playlist from first to last: Perfect -> Shape of You -> Believer -> Faded
1. Add song at beginning
2. Add song at end
3. Insert song after another song
4. Remove first song
5. Display playlist
6. Show song count
0. Exit
Enter your choice: 4


practical5_1.exe
Remove first song: Perfect
Total songs: 3
Playlist from first to last: Shape of You -> Believer -> Faded
1. Add song at beginning
2. Add song at end
3. Insert song after another song
4. Remove first song
5. Display playlist
6. Show song count
0. Exit
Enter your choice: 6
Total songs: 3
1. Add song at beginning
2. Add song at end
3. Insert song after another song
4. Remove first song
5. Display playlist
6. Show song count
0. Exit
Enter your choice: 3
Enter the existing song name: Imagine
Enter the new song name: Heaven
Song 'Imagine' not found. Insertion cancelled.



Problem 2:

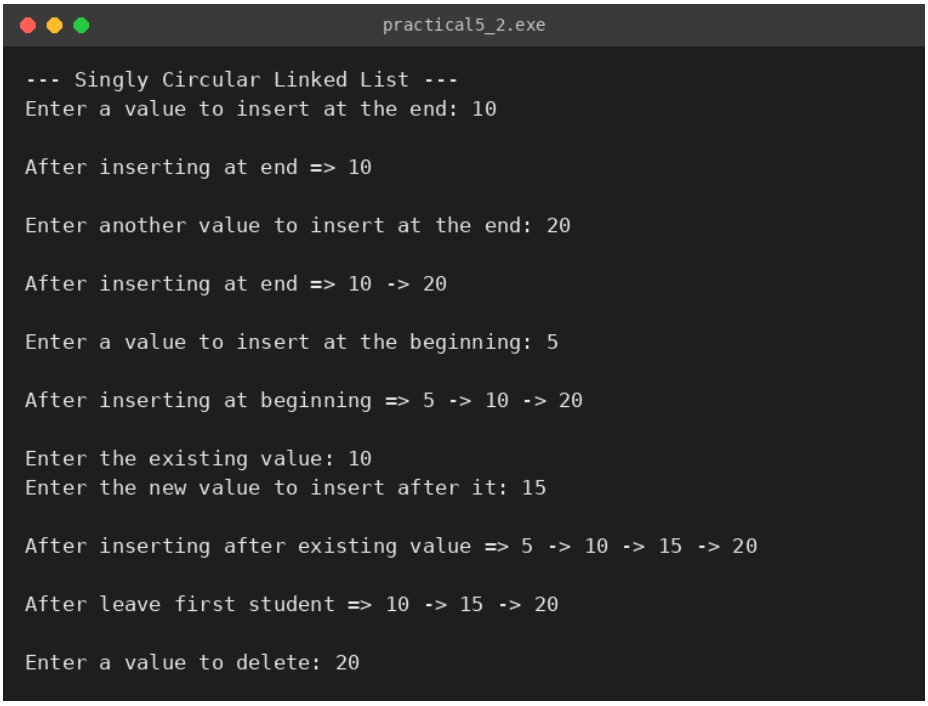
A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation.

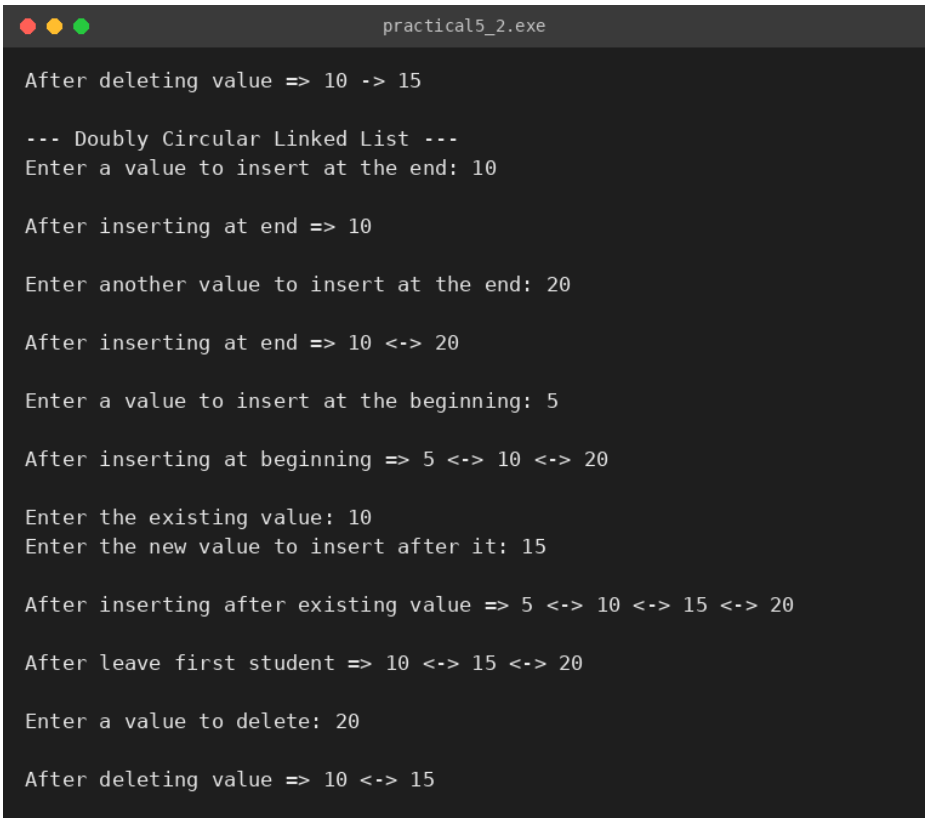
Approach: Both versions keep a head pointer, and the last node always links back to head (and in the doubly circular version head->prev is the last node). Every operation rewires the neighbouring links so that the loop is closed again before the function returns. When a student leaves, the previous node is linked to the next node; if the head leaves, the last node is also re-pointed to the new head. The case most likely to break the circle is removing the first (head) node or the only node. The code handles these explicitly: for the head it re-points the last node to the new head (last->next = newHead; in the doubly version the last node is directly available as head->prev, and newHead->prev = last is also updated), and for a single node it deletes it and sets head to nullptr.

For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-5/practical5_2.cpp

Output :





KEY QUESTIONS:

Problem 1: A doubly linked list maintains two links per node instead of one. When deleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

Problem 2: If your traversal of the circular list does not stop at the right moment, it will loop forever. Without referencing any specific code, explain what condition must be true at the moment you stop, and why that condition would fail if you set the last node's link to null.

ANSWERS:

ANS-1: In a doubly linked list the second node's prev pointer holds the address of the front node. When the front node is deleted, head is moved to the second node, but that node's prev still points to the freed front node. So the new head must be explicitly given prev = nullptr (and if it was the only node, tail must also be set to nullptr). In a singly linked list nothing points backwards, so moving head forward is enough.

If this step is forgotten, the new first node holds a dangling pointer to deleted memory. Any later backward traversal, or any deletion of that node that does prev->next = ..., will access freed memory, which leads to undefined behaviour such as a crash or corrupted playlist.

ANS-2: The traversal must stop at the moment we arrive back at the node where we started (temp == head, or temp->next == head for the last node), i.e. after every node has been visited exactly once. A circular list has no null end, so a check such as 'next == null' can never become true and would loop forever.

If the last node's link were set to null, the list would no longer be circular: the 'back at head' condition could never become true, because the traversal would run off the end and reach null instead of head. A loop written as 'do { ... } while (temp != head)' would then dereference a null pointer and crash, and the passing game would also be broken, since the last student could not pass the token back to the first.